Category 1: General Analytics & Business Logic

1. Difference between Data Analytics and Data Science

- **Data Analytics:** Focuses on analyzing existing data to find insights, trends, and support decision-making. Uses tools like SQL, Excel, Power BI.
- **Data Science:** Broader field that includes analytics plus **machine learning, predictive modeling, and advanced algorithms** to build models and make predictions.

---

2. The 4 Types of Analytics

1. **Descriptive Analytics:**
   Explains **what happened**.
   Example: Sales report showing revenue last month.

2. **Diagnostic Analytics:**
   Explains **why it happened**.
   Example: Analyzing why sales dropped in a certain region.

3. **Predictive Analytics:**
   Predicts **what will happen in the future** using models.
   Example: Forecasting next month's sales.

4. **Prescriptive Analytics:**
   Suggests **what actions should be taken**.
   Example: Recommending the best pricing strategy to increase revenue.

---

3. Structured vs. Unstructured Data

- **Structured Data:** Organized in tables with rows and columns.
  Example: Customer database, sales transactions in SQL tables.
- **Unstructured Data:** Not organized in a fixed format.
  Example: Emails, images, videos, social media posts, text documents.

---

4. KPI Design (SMART Framework)

A good KPI should be **SMART**:

- **Specific:** Clearly defined metric
- **Measurable:** Can be quantified with data
- **Achievable:** Realistically attainable
- **Relevant:** Linked to business goals
- **Time-bound:** Measured within a specific time period

Example:
"Increase monthly website conversions by **10% within 3 months**."

---

5. Translating a Vague Business Problem into a Data Project

Steps:

1. **Clarify the business goal** (What is the company trying to improve?)
2. **Define the key metric** (What will measure success?)
3. **Formulate analytical questions**
4. **Identify required data sources**
5. **Choose the analysis or model approach**
6. **Deliver insights and recommendations**

Example:
Problem: "Sales are low."
→ Question: Which products, regions, or time periods have declining sales?

---

6. The "So What?" Test (Metric Drop Investigation)

1. **Confirm the data is correct** (check for data errors).
2. **Identify when the drop started.**
3. **Segment the metric** (by region, product, customer type, etc.).
4. **Look for anomalies or changes** (marketing campaigns, pricing, seasonality).
5. **Compare with historical data.**
6. **Form hypotheses and test them.**
7. **Provide insights and recommended actions.**

---

7. Data Grain (Granularity)

- **Granularity** refers to the **level of detail in the data**.

Example:

- Daily sales per store → **fine granularity**
- Monthly total sales → **coarse granularity**

Why it matters when joining data:
If datasets have different granularity (e.g., daily vs monthly), joining them incorrectly can cause **duplicate rows, incorrect totals, or misleading analysis**.

Category 2: Statistics & Probability

Central Tendency:

- **Mean:** Average of all values.

- **Median:** Middle value when data is sorted.

- **Mode:** Most frequent value.
  Use **Median** instead of Mean when data has **outliers or skewed distribution** (e.g., income data).

Spread:

- **Variance:** Measures how far data points are spread from the mean.

- **Standard Deviation:** Square root of variance; shows how much values typically deviate from the mean in the same unit as the data.

Correlation vs. Causation:
Correlation means two variables move together, but it doesn't mean one causes the other.
Example: Ice cream sales and drowning incidents both increase in summer. The real cause is **hot weather**, not ice cream.

Probability Distributions:
A **Normal Distribution** is a bell-shaped curve where most values are around the mean and fewer appear at the extremes.
It is important because many natural and business phenomena (heights, measurement errors, demand patterns) follow this distribution.

Hypothesis Testing:

- **P-value:** Probability that the observed result happened by chance. A small p-value (e.g., <0.05) suggests the result is statistically significant.

- **Confidence Interval:** A range of values where the true result likely lies (e.g., 95% confidence means we are 95% sure the true value is within that range).

Error Types:

- **Type I Error (False Positive):** Detecting an effect when none exists.

- **Type II Error (False Negative):** Missing a real effect.
  In **medical contexts**, a **Type II error** can be worse because it may miss a real disease.

A/B Testing:
Split users into two groups: A (control) and B (new change). Measure a key metric (e.g., conversion rate) and use statistical testing to check if the difference is **significant (p-value < 0.05)**.

Regression Analysis:
Used to **model the relationship between variables** and predict outcomes (e.g., predicting sales based on advertising spend).
Main assumptions: linear relationship, independence of errors, constant variance (homoscedasticity), and normally distributed errors.

Category 3: SQL (The Technical Core)

The Basics (WHERE vs HAVING):

- **WHERE:** Filters rows **before aggregation**.
- **HAVING:** Filters groups **after GROUP BY aggregation**.

DELETE vs DROP vs TRUNCATE:

- **DELETE:** Removes rows, table structure stays.
- **TRUNCATE:** Removes all rows quickly, cannot filter with WHERE.
- **DROP:** Deletes the entire table structure and data.

Multiple Primary Keys:
A table cannot have multiple primary keys, but it can have **one composite primary key** (multiple columns together).

Foreign Key NULL:
Yes, a **foreign key can contain NULL** if the relationship is optional.

Composite Key:
A key made of **two or more columns together** used to uniquely identify a row.

UNIQUE vs PRIMARY KEY:

- **PRIMARY KEY:** Unique + NOT NULL, only one per table.
- **UNIQUE:** Ensures uniqueness but can allow NULLs and multiple per table.

Subqueries:
A query inside another query used to filter or calculate results.

Joins:

- **INNER JOIN:** Returns only matching rows from both tables.

- **LEFT JOIN:** Returns all rows from the left table and matching rows from the right.

Primary Key vs Foreign Key:

- **Primary Key:** Unique identifier for each row in a table.
- **Foreign Key:** Column that references a primary key in another table.

GROUP BY vs ORDER BY:

- **GROUP BY:** Groups rows to apply aggregate functions (SUM, COUNT).
- **ORDER BY:** Sorts the result set.

Deduplication:
Find duplicates:

```
SELECT column, COUNT(*)
FROM table
GROUP BY column
HAVING COUNT(*) > 1;
```

Remove duplicates using **ROW_NUMBER()** and delete rows where row_number > 1.

UNION vs UNION ALL:

- **UNION:** Removes duplicates.
- **UNION ALL:** Keeps duplicates and is faster.

Window Functions:

- **ROW_NUMBER():** Unique sequential number per row.
- **RANK():** Same rank for ties with gaps.
- **LEAD/LAG:** Access next or previous row values.

Products never sold:

```
SELECT p.product_id
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id
WHERE o.product_id IS NULL;
```

Customers never ordered:

```
SELECT c.customer_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

CTE (Common Table Expression):
Temporary result set defined with **WITH**. It improves readability and can simplify complex queries compared to subqueries.

Indexes:
A data structure that speeds up searching in a table (like a book index).

How indexes speed queries:
They allow the database to **find rows quickly without scanning the whole table**.

Clustered vs Non-Clustered Index:

- **Clustered:** Determines the physical order of data in the table (only one).
- **Non-Clustered:** Separate structure that points to the data (multiple allowed).

Hashed Index:
Uses a hash function to map keys to locations, very fast for **exact matches**.

When NOT to use an index:
When tables are very small, columns change frequently, or the column has low uniqueness.

Query Execution Plan:
A plan generated by the database showing **how the query will be executed** (which indexes, joins, scans are used).

Top 3 products by sales per category:

```
SELECT *
FROM (
SELECT product_id, category_id, SUM(sales) AS total_sales,
ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY SUM(sales) DESC)
r
FROM sales
GROUP BY product_id, category_id
) t
WHERE r <= 3;
```

Running total of sales by date:

```
SELECT date,
SUM(sales) OVER (ORDER BY date) AS running_total
FROM sales;
```

First and last purchase per customer:

```
SELECT customer_id,
MIN(order_date) AS first_purchase,
MAX(order_date) AS last_purchase
FROM orders
GROUP BY customer_id;
```

Employees earning more than average:

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Category 4: Power BI & DAX

Row Context vs Filter Context:

- **Row Context:** Calculation happens **row by row** (often in calculated columns).
- **Filter Context:** Filters applied to data before a measure is calculated (from slicers, filters, visuals).

Calculated Column vs Measure:

- **Calculated Column:** Computed **for each row** and stored in the table.
- **Measure:** Calculated **dynamically during analysis** using aggregation.
  Use **Measures** when calculations depend on filters or aggregations.

Drill Down vs Drill Through:

- **Drill Down:** Navigate **within the same visual hierarchy** (e.g., Year → Month).
- **Drill Through:** Navigate to **another report page with detailed data**.

Incremental Refresh:
A Power BI feature that refreshes **only new or changed data instead of the entire dataset**, improving performance.

Why CALCULATE is important:
Because it **modifies filter context** and evaluates expressions under different filters, making it the core of advanced DAX calculations.

ALL() vs ALLSELECTED() vs REMOVEFILTERS():

- **ALL():** Removes all filters from a table or column.
- **ALLSELECTED():** Keeps filters from user selections but ignores visual-level filters.
- **REMOVEFILTERS():** Explicitly removes filters similar to ALL but clearer in intent.

VALUES() vs DISTINCT():

- **VALUES():** Returns unique values and may include a **blank row** for missing relationships.
- **DISTINCT():** Returns only unique values without special blank behavior.

Import Mode vs DirectQuery vs Composite:

- **Import Mode:** Data loaded into Power BI memory (fast performance).
- **DirectQuery:** Queries data directly from the source each time (no storage in Power BI).
- **Composite Model:** Combination of Import and DirectQuery.

Scheduling Refresh (Power BI Service):
Publish dataset → Go to **Settings** → **Dataset** → **Scheduled Refresh** → Set refresh frequency and credentials.

Gateway & On-Premises Data Gateway:
A **gateway** allows Power BI Service to securely access **on-premise data sources** like local databases.

Row-Level Security (RLS):
A security feature that restricts data access **based on user roles**.

Implementing RLS:
Create roles in **Model View** → **Manage Roles** → **Define filters** (e.g., region = "Egypt").

Dynamic RLS:
Uses user identity (like USERPRINCIPALNAME()) to automatically filter data for each logged-in user.

Testing RLS:
Use **"View as Role"** in Power BI Desktop to simulate user access.

Object-Level Security (OLS):
Restricts access to **specific tables or columns** in the data model.

Securing Sensitive Columns:
Use **OLS** or remove/hide sensitive columns from the report model.

Data Modeling Designs:
Common design is **Star Schema**:

- Fact table (metrics)
- Dimension tables (descriptive attributes).

Working with 2 Fact Tables:
Connect both fact tables to **shared dimension tables** (conformed dimensions).

Implementing RLS in model:
Apply filters on **dimension tables** so they propagate to related fact tables.

Filter vs Slicer:

- **Filter:** Applied in filter pane, affects visuals/pages.
- **Slicer:** Interactive visual users use to filter report data.

Conditional Column vs Custom Column (Power Query):

- **Conditional Column:** Built using rules like IF conditions through UI.
- **Custom Column:** Uses **M language formula** for more advanced logic.

Executive vs Operational Dashboard:

- **Executive Dashboard:** High-level KPIs, simple visuals, strategic insights.
- **Operational Dashboard:** Detailed metrics, real-time monitoring, more granular data.

Category 5: Python & Data Cleaning

How Python helps:
Python helps in **data analysis, automation, machine learning, and data cleaning**. It allows analysts to process large datasets, build models, and create visualizations efficiently.

Python Scripts:
A Python script is a **file containing Python code (.py)** that automates tasks such as data processing, analysis, or reporting.

Data Movement with Python:
Python can move data between systems using libraries like **Pandas, SQL connectors, APIs, and file handling** (CSV, Excel, JSON).

Integrating SQL with Python:
Use libraries like **sqlite3, psycopg2, or SQLAlchemy** to connect Python with databases, run SQL queries, and load results into Pandas DataFrames.

List vs Tuple:

- **List:** Mutable (can be changed), uses `[ ]`.
- **Tuple:** Immutable (cannot be changed after creation), uses `( )`.
  Tuples are immutable to ensure **data integrity and faster performance**.

Common Libraries:

- **Pandas:** Data analysis and manipulation.
- **NumPy:** Mathematical and numerical operations.
- **Matplotlib / Seaborn:** Data visualization.

Handling Missing Values and Outliers:

- **Missing values (NaN):** Fill with mean/median, forward fill, or remove rows.
- **Outliers:** Detect using statistical methods (IQR, Z-score) and remove or cap them.

Pandas groupby() and merge():

- **groupby():** Used to aggregate data by categories (e.g., sales by region).
- **merge():** Combines two DataFrames based on a common column (similar to SQL join).

Category 6: Data Modeling & Databases

Star Schema vs Snowflake Schema:

- **Star Schema:** Fact table in the center with **denormalized dimension tables** (simpler and faster for queries).
- **Snowflake Schema:** Dimension tables are **normalized into multiple related tables** (more complex but reduces redundancy).

Many-to-Many Relationships:
Handled using a **bridge (junction) table** that connects the two tables and resolves the many-to-many relationship.

View vs Materialized View:

- **View:** Virtual table created from a query, data is not stored.
- **Materialized View:** Stores the query result physically, so it is faster but needs refresh.

ETL vs ELT:

- **ETL:** Extract → Transform → Load (data transformed before loading to the warehouse).
- **ELT:** Extract → Load → Transform (data loaded first, then transformed inside the warehouse).

Behavioral & Case Study Answers

Tell me about yourself:
I am a Data Science student focusing on data analytics. I work with tools like SQL, Python, and Power BI to analyze data and build dashboards. I enjoy turning raw data into insights that help businesses make better decisions.

Why data analytics:
Because it combines **problem solving, business understanding, and technical skills** to discover insights and support data-driven decisions.

What data problems you enjoy:
Problems related to **finding patterns, improving business performance, and optimizing processes** using data.

Business Impact

Project influencing decision:
I built a **Power BI dashboard analyzing sales trends**, which helped identify underperforming products and guided marketing focus.

Reducing cost / increasing revenue:
By analyzing sales data and customer behavior, I identified products with low margins and suggested pricing adjustments.

Measuring success:
Through **KPIs such as revenue growth, conversion rate, or cost reduction** after implementing the recommendations.

Insights not obvious at first:
After deeper segmentation analysis, I found that a drop in sales was caused by a **specific region and product category**, not the whole market.

Challenging an assumption:
Data analysis showed that a product thought to be low-performing actually had **high sales during specific seasons**.

Communication

Explaining complex data:
I simplify insights using **visualizations, clear examples, and business-focused explanations** instead of technical terms.

Presenting to executives:
Focus on **key KPIs, trends, and actionable insights**, avoiding unnecessary technical details.

Choosing detail level:
Executives get **high-level KPIs**, while analysts or technical teams receive **more detailed data and methodology**.

Storytelling with data:
I structure insights as **problem → analysis → key finding → business recommendation**.

Making dashboards actionable:
Include **clear KPIs, filters, and insights that suggest specific actions** rather than only showing numbers.

Prioritizing stakeholder requests:
Based on **business impact, urgency, and available resources**.

Problem Solving

Most challenging data problem:
Cleaning and integrating data from **multiple inconsistent sources** before building a reliable analysis model.

Ambiguous problems:
Start by **clarifying the goal, exploring available data, and forming hypotheses** to guide analysis.

Incomplete data:
Use the available data, make **assumptions clear**, and provide recommendations with stated limitations.

Validating assumptions:
Through **data checks, statistical testing, and comparing results with historical trends**.

Time Management

Handling tight deadlines:
Break the problem into **smaller tasks, focus on key metrics, and deliver the most important insights first**.

Delivering insights quickly:
Use **quick exploratory analysis and simple visualizations** to identify major patterns.

Balancing speed vs accuracy:
Deliver **initial insights quickly**, then refine analysis for higher accuracy.

Changing priorities:
Reassess tasks and focus on the **highest business impact items**.

Handling repetitive requests:
Automate tasks using **SQL queries, Python scripts, or dashboards** to reduce manual work.